百亿补贴 C 端 AI Coding 实战：基于 SDD 的服务端 AI Coding 实践

原创 天猫技术 大淘宝技术 2026年7月27日 01:49 浙江 38人



本文针对服务端AI Coding在复杂业务场景中难以达到生产级质量的问题，提出并实践了基于SDD（规格驱动开发）的解决方案。通过摒弃完全自动化的端到端流程，转而采用“开发者深度参与”结合Aone Copilot工具链的模式，将Spec作为唯一真相源以解决AI的概率性输出、上下文限制及局部视野瓶颈；同时引入自进化记忆机制（topic-open/save）管理跨会话上下文，确保知识沉淀与风格一致。最终在百亿补贴项目中实现了0手写代码的高质量交付，证明了SDD方法论配合开发者对Spec定义、架构审核及质量把控的深度介入，是将AI Coding从实验性工具转化为可控、可复制的生产力关键。

01 先说结果

从Copilot到Cursor，从代码补全到Agent自主编码，AI Coding工具正在飞速进化。但在真实的业务项目中，面对历史包袱、复杂交互和线上质量要求，我们要如何才能让AI Coding真正落地？

以下是我们给出的答案：

实践案例：百亿补贴某某玩法迭代，支持分享回流

需求规模：4k+代码行数

主要挑战：1年前历史项目迭代；玩法交互和逻辑复杂；测试时间有限，尽量降低回归成本

交付结果：0手写代码，Spec行数：5000+，长期记忆：200+，任务记录：500+，代码行：4000+

基于SDD方法论，我们实现了0手写代码、4天完成开发（不含联调）。Spec驱动带来的不只是效率提升，更是质量可控、风格一致、产出可追溯、和方法可复制。

02 AI Coding方法论演进

指导思想：SDD—从混乱到可控

我们的核心观点：深刻认同并**严格遵循当前AI Coding的事实标准——Spec-Driven Development（SDD）**。

这不是因为我们选择了某种"流派"，而是AI的技术特性决定了：没有Spec的AI Coding必然失控，只有Spec才能建立长期可控的人机协作。

AI Coding的天然瓶颈

在深入方法之前，必须理解AI写代码的三个硬约束：

- **概率性输出**：同样的提示词，两次运行结果可能完全不同
- **上下文限制**：对话轮次多了，模型会"遗忘"早期的约定；上下文压缩后，关键约束容易丢失
- **局部视野**：大项目里，AI只能看到你提供的部分代码，基于局部信息做的修改很可能破坏全局架构

这些不是模型不够强，而是当前AI的固有特性。**任何不针对这些限制设计的方法论，最终都会撞墙。**

为什么Vibe Coding走不通

Vibe Coding（完全用自然语言描述，让AI自由发挥）在小项目、原型阶段也许可行，但在实际业务中必然失败：

核心问题：没有标准和约束

- **迭代失控**：第一次迭代和第五次迭代，代码风格可能完全不同；没有约束的情况下，AI会随意修改之前的架构决策
- **上下文坍塌**：多轮对话后，模型开始"失忆"，重复犯之前纠正过的错误
- **局部冲突**：基于当前可见的几千行代码做的"优化"，可能和项目其他部分的逻辑直接冲突

对于有经验的开发者来说，这从一开始就是不可接受的。我们需要的不是"可能能跑"的代码，而是"可预测、可维护、可协作"的工程产物。

SDD：建立唯一的真相源

SDD（Spec–Driven Development）的核心理念是：**代码是Spec的副作用**。我们将规格说明（Specification）作为唯一真实来源（Single Source of Truth），代码只是它的派生产物。

这样做直接解决了上面的三个问题：

- **解决概率性**：Spec作为"唯一真相源"，每次AI实现都基于同一份确定性文档，而不是基于概率性的对话历史
- **解决上下文限制**：Spec是对整个系统的压缩表示。10万行代码的Spec可能只有几千字，AI能一次性读完并理解全局约束
- **解决局部冲突**：修改任何功能前，先更新Spec；Spec定义了全局架构和约束，AI基于更新后的Spec生成代码，自然保持全局一致性

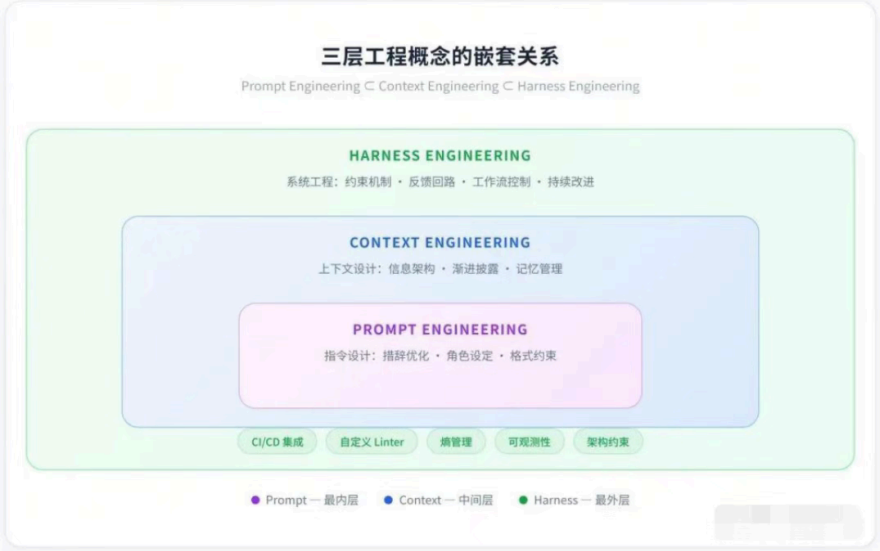
- 我们的实践框架

我们采用了标准的SDD流程：**Specify（规格定义）–> Plan（方案规划）–> Implement（代码实现）–> Validate（验证确认）**。具体会在后面的实践章节详细展开。

为了落地SDD，我们借鉴了Spec Kit等业界实践的思想，建立了**分层的文档体系**：

- **需求层**：定义"做什么"和"为什么做"，明确功能边界和"不做"的范围（Non–Goals）
- **架构层**：基于需求生成技术方案，包含模块划分和接口约定，由开发者审核把关
- **执行层**：将方案拆解为可独立验证的原子任务，确保每个任务有明确的交付标准
- **约束层**：定义项目级的"宪法"，包括编码规范、安全底线、技术栈限制等元规则，所有需求都必须遵守

- 与Harness Engineering的关系



SDD是Harness Engineering（驾驭工程）在AI Coding领域的具体实践。Harness解决的核心问题是：**当AI能力足够强时，如何防止它失控**。

我们将SDD映射到Harness的四层架构中：

- **约束层**：定义不可违背的技术原则，防止AI随意变更架构决策
- **上下文层**：分层文档体系（需求→架构→任务）确保AI获得"恰好足够"的信息，避免过载或缺失
- **执行层**：Specify→Plan→Implement→Validate的强制流程，防止"一股脑写代码"或"过早宣布胜利"
- **记忆层**：Spec作为持久化的唯一真相源，解决跨会话的"失忆"和状态漂移问题

核心结论：模型能力不是瓶颈，**SDD**才是让AI Coding从"碰运气"变成了**可重复、可管控的关键工程**。

历史尝试：多Agent P2C工作流 + Coding Agent

具体到SDD的实施，在早期实践中，我们建设了一套多Agent P2C（PRD to Code）工作流。

- 核心思路

整体目标：“端到端”、全自动P2C

整体步骤：**预生成知识库** –> **需求分析&改写** –> **技术方案生成** –> **代码生成** –> 自动化测试（规划）。每一步都产出下一步的Spec。

分步流程：具体每一层，又基于**软件工程学**，分别设计了**多个Agent**参与。拿需求分析&改写举例：**流程总控**（coordinator）–> **内容生成**（rewriter） –> **问题澄清**（clarifier） –> **内容校验**（analyzer）。校验不通过最多迭代3次。

> 虽然目标是“端到端”、全自动，但是不得不提供最小化问答式问题澄清。

- 实际结果

什么能工作：

- 最后能实际生成知识库、需求Spec、技术方案Spec、最终代码，采纳率约70%。

什么不符合预期：

- 内部步骤过多，单次任务执行时间偏长，严重影响反馈和调整实时性

- 有限人机交互无法彻底解决Spec不精准的问题，问题向后传递，影响最终采纳率
- 我们先后在iFlow CLI和Claude Code里实施，缺少成熟的代码库索引和Linter方案（特别是阿里级别的Java应用，使用LSP很快OOM），影响最终采纳率

- 最终结论

虽然能产出代码，但追求完全自动化的路线存在明显不足，难以达到稳定生产要求。这让我们确认：

- 整体流程设计必须考虑**Spec快速迭代**能力
- **关键节点质量**必须由开发者把控
- 复杂项目，依然需要开发者参与**上下文工程**（比如手动压缩与总结、手动开启新会话）

当前方案：Aone Copilot + 开发者深度参与的SDD

从P2C工作流的经验中，我们意识到一个关键问题：AI Coding的瓶颈不在代码生成，而在上下文管理和质量把控。全自动流水线看起来很美，但每一步的误差会层层放大，最终导致产出不可控。 因此，我们升级了实践方案：不再追求"端到端自动化"，而是让开发者回到驾驶座，用Aone Copilot + 开发者深度参与的方式落地SDD。

- 为什么选择Aone Copilot

从P2C的教训来看，工具链的成熟度直接决定了AI Coding的天花板。相比于Coding Agent等CLI工具，Aone Copilot作为IDE Extension具备几个关键优势：

- **内置Linter**：和IDE深度集成，语法错误、类型不匹配等低级问题在生成时就被拦截，不用等到编译才发现
- **代码库索引**：自带语义检索能力，AI能高效定位现有代码，不再"盲人摸象"
- **代码可见**：所有AI的修改实时呈现在编辑器中，开发者可以逐行审查，随时介入——这正是我们说的"开发者深度参与"的基础
- **Agent能力成熟**：Aone Copilot本身的Agent和上下文工程足够优秀，省去了大量自建基础设施的成本

简单说，**工具的成熟度决定了我们能把精力花在"思考"上，而不是花在"和工具搏斗"上。**

- 流程变化：从全自动到精确控制

P2C追求的是"一键到底"，而新方案的核心变化是：**开发者精确控制Spec的每一步演进**。

具体来说：

- **不重复造轮子**：依赖Aone Copilot主Agent、内置MCP、大模型本身能力，而不是自建一套复杂的多Agent编排系统。
- **放弃全自动、放弃长任务**：不再让AI一口气跑完整个流程，而是开发者逐步推进。特别是项目初期，每一步Spec的产出都经过人工审核后才进入下一步
- **灵活的文件策略**：不强制使用Spec Kit等特定工具的目录结构，但严格遵循"先更新Spec，再生成代码"的原则。能力变更时，也是先迭代Spec，再自动生码

这个转变的本质是：**从"信任AI的全流程判断"变成"信任AI的单步执行能力"**。AI擅长在明确约束下高质量地完成单个任务，但不擅长在长链路中自主做出一连串正确的决策。

- 面临的挑战

方案确定了，但落地时还有两个绕不开的挑战：

挑战一：记忆管理

一个需求可能要跨多个会话才能完成。每次新开会话，AI就"失忆"了——之前讨论过的架构决策、踩过的坑、确认过的约束，全部丢失。你跟AI说"帮我优化一下支付模块"，AI一脸懵："支付模块在哪？用了什么技术栈？有什么历史包袱？"于是你又要花半小时重新"自我介绍"。另外，无关模块间的记忆隔离（节省上下文）、多人研发中的记忆同步也是需要解决的问题。

挑战二：质量保证

阿里的项目基本上是分布式架构，部署和集成测试的成本很高——甚至本地启动和单元测试都有不小的门槛。如果用mock来降低测试成本，又无法覆盖真实场景。基于TDD的Superpowers Skill，在这种环境下也失去用武之地。

这些挑战如何解决？我们将在实践过程中逐一展开。



整体流程

我们遵循SDD的标准四步流程：**Specify → Plan → Implement → Validate**，但没有死板地套用某个框架的文件结构。实际的Spec组织方式是根据项目需要自然演化出来的：



其中，经AI整理、人工补充后的PRD+现有实现调研+问题澄清，等同于`spec.md` + `constitution.md`，技术方案+api文档，则等同于`plan.md`，`tasks.md`放在临时目录里，用完即弃。

本质上还是SDD的分层思路——需求层、架构层、执行层、约束层——只是文件命名和组织方式更贴合我们的实际工作习惯。

- Specify（规格定义）

这一步我们做了三件事：

PRD分析与理解：拿到产品的PRD后，用AI辅助做结构化整理——提取核心功能点、识别隐含的技术约束、明确Non-Goals。特别是Non-Goals，**不写清楚"不做什么"，AI一定会自由发挥**。

参考代码索引：刮刮乐是1年前的历史项目，不能让AI在整个代码库里大海捞针。我们提供了参考代码的入口，再由AI自主梳理现有模块的职责和依赖关系，把这些整理进Spec里。**本质是缩**

小AI的搜索空间。

约束补充：技术栈限制、编码规范、安全底线、性能要求——这些"显而易见"的东西必须显式写进Spec。对AI来说，**没写下来的规则就是不存在的规则**。特别是针对玩法项目（因为玩法逻辑复杂），我们额外强调了两条约束：

- **注释齐全**：对AI和开发者都友好的注释，提升代码的可维护性
- **日志齐全**：方便测试阶段排查问题，甚至上线后的线上问题定位

整个Specify过程可以抽象成Skill或Agent，但我们的经验是：**一个好的提示词模板就够了**，过度工程化反而增加维护成本。

- Plan（方案规划）

技术资产调研：让AI带着明确的问题去读代码。比如："现有的任务系统是否支持单日助力上限？扩展点在哪里？"——而不是让它漫无目的地"了解项目"。

问题澄清：让AI主动提出它发现的疑问和风险点——需求中有没有遗漏？有没有矛盾？现有代码有没有和新需求冲突的设计？**把问题暴露在方案阶段，而不是编码阶段**。

技术方案产出：AI生成技术方案，包含模块划分、接口约定、数据流设计和任务拆解。**开发者的核心职责是审核架构合理性**——AI给的方案"能跑"，但不一定"好"，比如它可能引入不必要的复杂度，或者忽略和现有架构的一致性。

- Implement（代码实现）

核心策略：**先搭框架，再填细节**。

第一步让AI搭建代码骨架——目录结构、模块接口、类型定义、基础数据流管道。产出是"能编译但没有具体实现"的框架。**框架必须由开发者审核确认**，因为后面所有代码都在这个骨架上生长，改框架的成本远高于改实现。

框架确认后，按任务列表逐模块实现。每个任务对应一个独立会话，开始前通过 `topic-open` 加载相关记忆和Spec片段，完成后通过 `topic-save` 保存关键决策。**一次只做一件事**，上下文越聚焦，产出质量越高。

- Validate（验证确认）

Validate不是最后才做的事——**它贯穿在每一步中**。每一步的输出Spec都必须和输入Spec对齐。

实际迭代次数：

- **技术方案与PRD对齐**：最初版本迭代了3次，后面调整规则和约束又迭代了2次
- **代码与技术方案对齐**：初始代码迭代了5次以上，后面技术方案和代码同步调整也超过了5次

验证手段：

- AI辅助对齐文档（对比Spec和方案的一致性）
- AI辅助对齐代码（对比代码和方案的一致性）
- 多模型交叉CR代码本身（利用不同模型的"认知差异"发现盲点）

> CR的重点不是"逻辑对不对"——逻辑层面AI很少犯错。重点是"设计好不好"：有没有过度抽象？错误处理是不是在吞异常？边界条件够不够健壮？跨模块的风格是否一致？

上下文工程

如果说SDD是"战略"，上下文工程就是"战术"。**给AI什么信息、给多少、什么时候给**，直接决定了每一次交互的质量。

- 核心原则：恰好足够

- **不能多**：上下文过长，AI会"迷失在中间"（Lost in the Middle）——关键约束可能恰好被淹没在中间位置
- **不能少**：信息不足时，AI会自信地"脑补"，给出看起来合理但实际错误的答案

- 工具支撑

实际上我们只用了**2个MCP工具 + 1套自建Skill**来管理上下文：

- **context7**：查询第三方开源库特定版本的文档和代码示例，不依赖AI可能过时的训练数据
- **artifact7**：面向阿里巴巴二方库的代码上下文治理。集团内部二方包普遍文档缺失，artifact7从SDK源码中解析接口定义和代码示例，通过精准版本绑定确保AI召回正确的API用法
- **topic-open / topic-save**：自进化记忆体（详见3.3），让每次对话都从"恰好足够"的上下文开始

- 实践技巧

文档预处理：不要每次都把原始PRD直接丢给AI。先让AI去除格式噪音、提取核心信息、按模块重新组织，后继使用处理后的文档，让时间和token花在"思考"上而不是"处理"上。

渐进式披露：实现模块A时，只加载模块A的Spec和它直接依赖的接口定义。需要全局视角时，加载架构概览而不是所有模块的详细设计。一次对话只涉及1–2个话题，信息聚焦，AI才能发挥最佳水平。

记忆管理：每完成一个里程碑立即 `topic-save`，只记有价值的内容——代码参考、解决方案、纠正过的错误。新任务开新会话，通过 `topic-open` 恢复长期记忆。通过 `tasks.md` 追踪短期进度，避免重复工作。

自进化记忆

针对2.3.3中的记忆管理挑战，我们开发了一个轻量级工具：**@ali/ai-coding-assistant**，核心是两个Skill——`topic-open` 和 `topic-save`。

- 设计思路

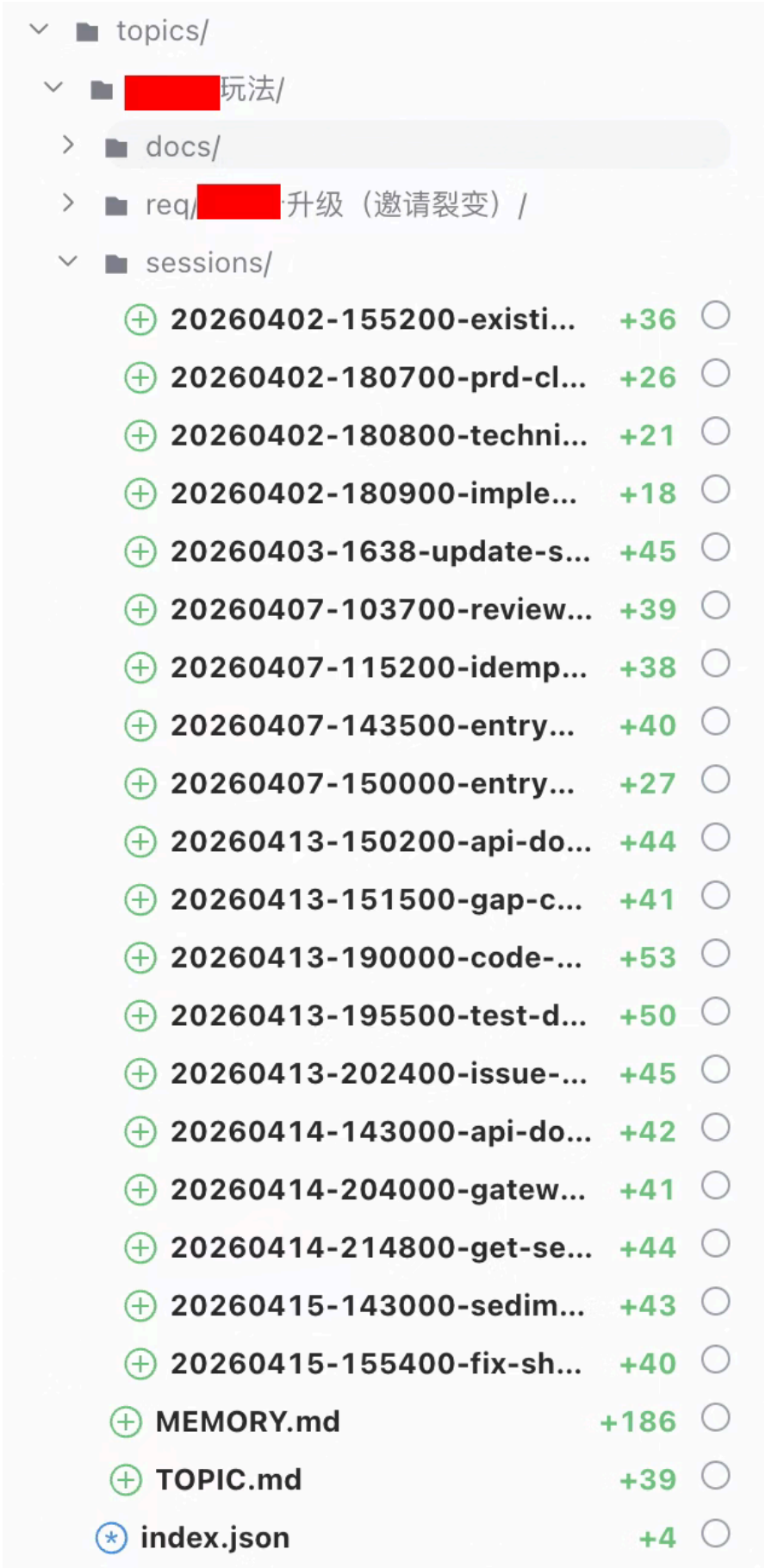
不做复杂的Agent系统，不改变现有工作流程，**只给AI装一个"记忆体"**。你聊过的需求、踩过的坑、做的技术决策，它都记住。下次打开这个话题，记忆自动加载到上下文里。

我们借鉴了OpenClaw的「md文件即知识」范式——所有记忆存成可读、可编辑、可Git版本化的 `.md` 文件。记忆不是静态的知识库，而是**随项目一起进化的活文档**。

- 核心设计：四层文件结构

```
topics/                          # 话题库
├── index.json                   # 第1层：话题索引，粗筛路由
└── <话题名>/
    ├── TOPIC.md                 # 第2层：话题定义，精确定位
    ├── MEMORY.md                # 第3层：跨会话技术决策总结
    └── sessions/                # 第4层：历史会话存档
        └── <时间戳>-<描述>.md
```

分层按需加载，节省上下文。第1层粗筛，第2层定位，第3层才是真正要加载的记忆，第4层平时不加载，只在追溯历史时查阅。



- 三个关键决策

- **话题隔离**：大项目拆成一个个话题，每个话题独立记忆，互不干扰。一次对话只涉及1–2个话题，避免"支付模块的记忆污染红包模块"
- **记忆即知识库**：记忆不是手动维护的文档，而是AI在对话中自动生成和进化的。只记有价值的内容——代码参考、解决方案、常见错误、跨话题关联
- **最小侵入**：不改变工作流程，不和现有工具冲突。对话开始时加载记忆，结束时保存记忆。两条核心规则：最小化改动；大的变更需要用户确认

- 效果

维度	使用前	使用后
精力成本	事前精心准备知识库，事后大量精力维护	自动保存/加载高价值记忆
需求迭代	每次都要让AI重新理解背景	经验无缝继承，迭代有记录
系统抗腐化性	多次迭代风格不一致，快速腐化	前面的约束后面每次都遵守
知识库时效性	知识库容易局部过期	记忆和话题绑定，和代码同步进化

这也回应了P2C阶段的教训：**不要用复杂的Agent编排来解决记忆问题**。精心编排的multi-agent系统，运行时长和token预算容易失控，和Aone Copilot的系统提示词可能冲突。而topic-open/topic-save利用现有系统能力，不干预主流程，反而更稳定。

同时，因为记忆文件是`.md`格式、存在项目目录中，天然支持Git版本化——团队成员拉取最新代码就能同步所有人的AI记忆，天然解决多人协作的问题。

质量保证

质量保证没有银弹。

分布式项目的部署成本摆在那里，mock覆盖不了真实场景，AI也无法替代人对业务逻辑的判断。我们的结论很务实——**当前阶段，质量保证必须靠人深度介入**。

具体做法：

- **多模型交叉CR**：用不同AI模型审查代码，利用模型间的"认知差异"发现盲点
- **AI辅助对齐**：让AI对比代码变更和Spec的一致性，辅助发现遗漏和偏差

- **开发者逐行CR**：重点关注设计合理性、边界条件和错误处理，不能因为"AI写的应该没问题"就放松
- **测试针对性回归**：不能只跑自动化用例，必须针对AI生成的代码做手工回归，特别是边界场景和异常路径

不优雅，但诚实。AI Coding提升了代码生成效率，但**质量保证的责任没有被转移——它依然在人的肩上。**



什么有效

Spec是真正的产出物，代码只是副产品。 把精力从"写代码"转移到"写Spec"上，整个开发过程的可控性有质的提升。Spec写得好，代码几乎是"自动"生成的。

开发者的角色从"写代码"变成"做决策"。 审核Spec、审核方案、审核架构——这些是高价值的决策工作。**AI负责执行，人负责判断。**

小步快跑优于一步到位。 任务拆小，每次只做一件事，做完立即验证。出了问题能精确定位是哪一步出的；让AI一口气写500行代码，出了bug都不知道从哪排查。

上下文工程是被严重低估的能力。 给AI提供什么上下文，比怎么写prompt重要10倍。同样的指令，在精心准备的上下文中和在混乱的上下文中，产出质量天差地别。

自进化记忆让迭代开发真正可行。 有了topic-open/topic-save，AI能跨会话保持一致的记忆，每次迭代都站在之前的经验之上。

什么需要改进

Spec膨胀：技术方案达到数千行，伪代码占据大量篇幅，很快打满上下文。违背了SDD“人定义WHAT，AI实现HOW”的核心原则。一旦发现“换一个技术栈实现，这份Spec是否仍然有效？”答案为否，就说明Spec里混入了HOW。应及时调整提示词禁止完整伪代码、按模块拆分Spec、建立抽象度检查环节。

对AI Coding的思考

AI Coding的上限不取决于模型有多强，而取决于**开发者的工程素养有多高**。懂架构、懂设计、懂如何拆解问题的开发者，能用AI产出远超个人能力的成果。

SDD的本质是把软件工程的最佳实践适配到人机协作的新范式中。需求分析、架构设计、模块拆解、质量验证——这些基本功在AI时代不但没过时，反而更重要了。因为**AI放大一切**：好的工程实践被放大成高效产出，差的工程实践也被放大成更大的混乱。

自进化记忆体让这套方法论从"每次从头开始"变成了"站在之前的肩膀上"。**SDD提供可控性，自进化记忆提供连续性**——两者结合，才是AI Coding真正能用于生产的关键。

回到开头的数据：4000+行代码，0手写，Spec驱动。**当方法论对了，AI Coding是真的能用于生产的。**

05 团队介绍

本文作者码一，来自淘天集团–天猫技术团队。

百亿补贴与聚划算，是淘天集团面向价格敏感用户和品牌商家的核心营销阵地，也是我们直面市场竞争最前沿的“主战场”。我们不断“刷新”技术能力，通过半托管、竞价和补贴系统的建设，支撑百亿补贴成为淘天过去三年里增长最快的业务。每天数千万用户在我们的场域完成“信任 – 比价 – 决策 – 下单”，背后是亿级流量、毫秒级响应、强一致性与极致稳定性的工程挑战。

当前，我们也正在用 AI 重构整个运营流程：从超链半托管、招商审核，到补贴，全面推动从“人工配置”向“AI 自治”演进。同时，我们也正积极探索 AI 对全栈研发流程的深度重构 ——从需求分析、接口设计、代码生成，到测试验证与部署上线，逐步构建端到端的 AI 原生开发范式。这里，是 AI 在电商营销场景落地的真实试验田，也是 AI 重塑软件工程本身的创新前沿。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 9765

留言 1

写留言



小白龙 浙江 7月27日 1

太强了。这确实是很了不起的aicoding实践和指导。之前ai生成全靠自己把握。让ai总结实现了什么改了什么。怎么实现的，手工测试，以及将实现生成和改进到readme文件里。特别原始人。